Assignment-01

NAME : P.L. DAVIKSHA

REG: NO: 192572001

COURSE CODE: CSA0644 Design and Analysis of algorithms for Modern computing

DATE: 9/4/2026.

Asymptotic Notations.

Big-O Notation (O)

It represents the upper bound of an algorithm's running time. It shows the maximum time an algorithm can take in the worst case

Eg

If an algorithm takes at most n^2 time, we write

$$T(n) = O(n^2)$$

Big-Omega Notation (Ω)

Big-Omega notation represents the lower bound of an algorithm's running time. It shows the minimum time required in the best case.

Eg

$$T(n) = \Omega(n)$$

Theta Notation (Θ)

It represents the tight bound of an algorithm. It shows the exact growth rate, meaning both upper and lower bounds are the same.

Eg

$$T(n) = \Theta(n^2)$$

Important Properties:-

Reflexivity.

A function is always bounded by itself.

$$f(n) = O(f(n)), \quad f(n) = \Omega(f(n)), \quad f(n) = \Theta(f(n))$$

Eg

$$n^2 = O(n^2)$$

Transitivity :-

If one function dominates another and that dominates a third, then:

If

$$f(n) = O(g(n)) \text{ and } g(n) = O(h(n))$$

then

$$f(n) = O(h(n))$$

Eg

$$n = O(n^2),\ n^2 = O(n^3) \Rightarrow n = O(n^3)$$

Symmetry (for Θ)

If $f(n) = \Theta(g(n))$, then

$g(n) = \Theta(f(n))$.

Constant Factor Rule

Constants are ignored in asymptotic analysis

$$c \cdot f(n) = O(f(n))$$

Eg

$$5n = O(n)$$

Addition Rule

When combining functions:

$$f(n) + g(n) = O(\max(f(n), g(n)))$$

Eg:-

$$n^2 + n = O(n^2)$$

Multiplication Rule

$$f(n) \cdot g(n) = O(f(n) \cdot g(n))$$

Eg

$$n \log n = O(n \log n)$$

Dominance Rule.

Higher growth dominates lower growth

Growth order:-

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \ldots 2^n < 3^n < \ldots < n^n$$

Mathematical Analysis of Towur of Hanoi.

Problem Overview:-

- 3 rods: Source, Auxiliary, Destination
- Move n disks
- Rules:
  - Move one disk at a time
  - No larger disk on smaller disk.

Recursive Function:-

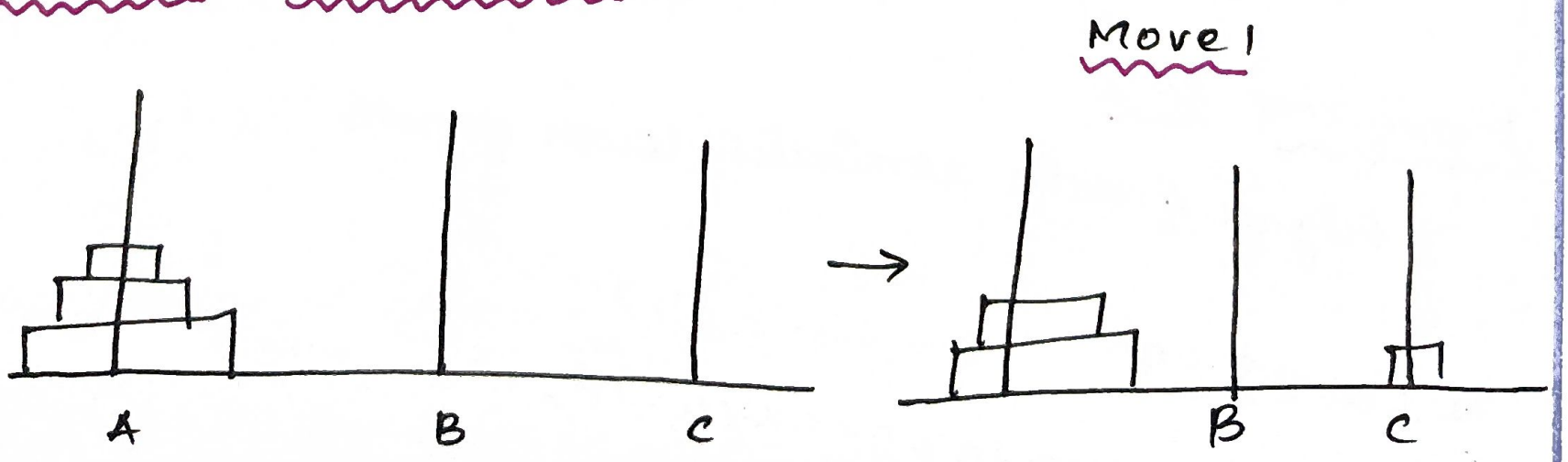
Let $T(n)$ = no. of moves required

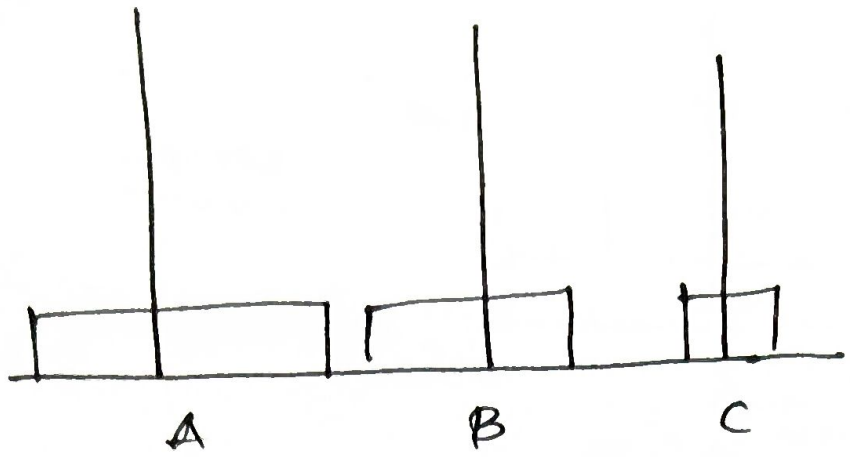
$$T(n) = 2T(n-1) + 1$$

with basecase:

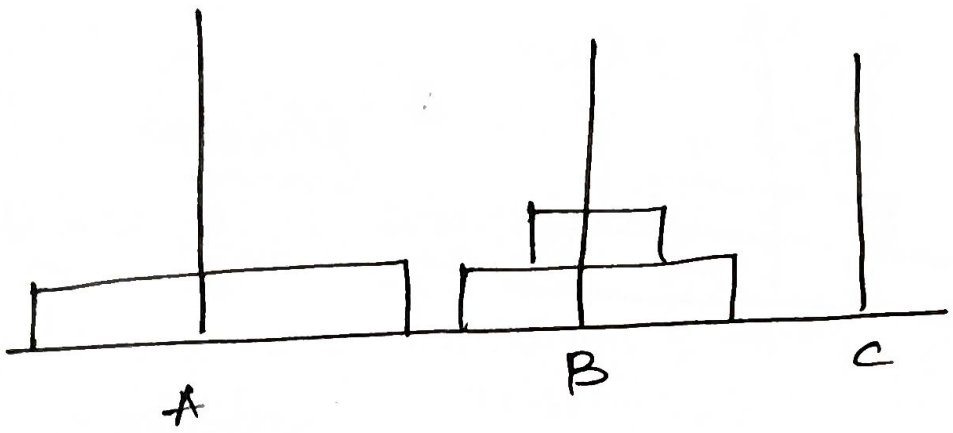
$$T(1) = 1$$

Recurrence Visualization:-

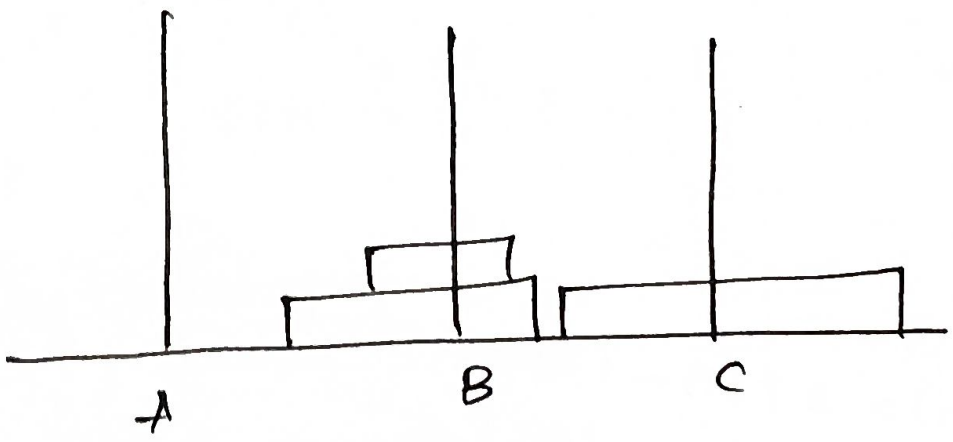




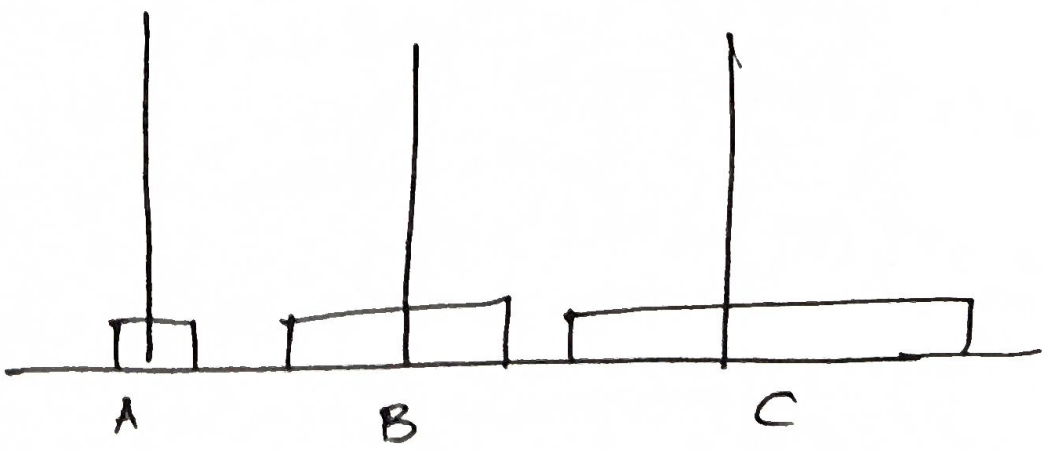
Move 2



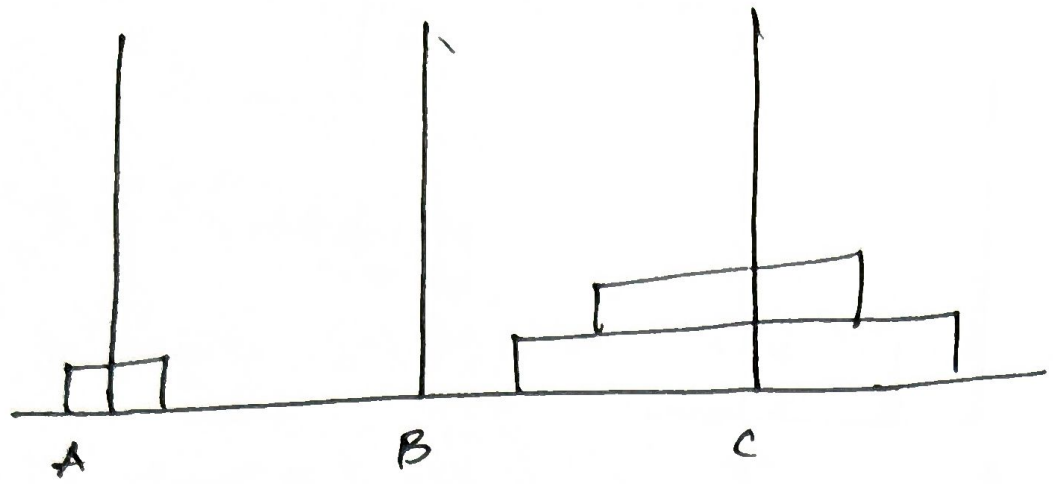
Move 3



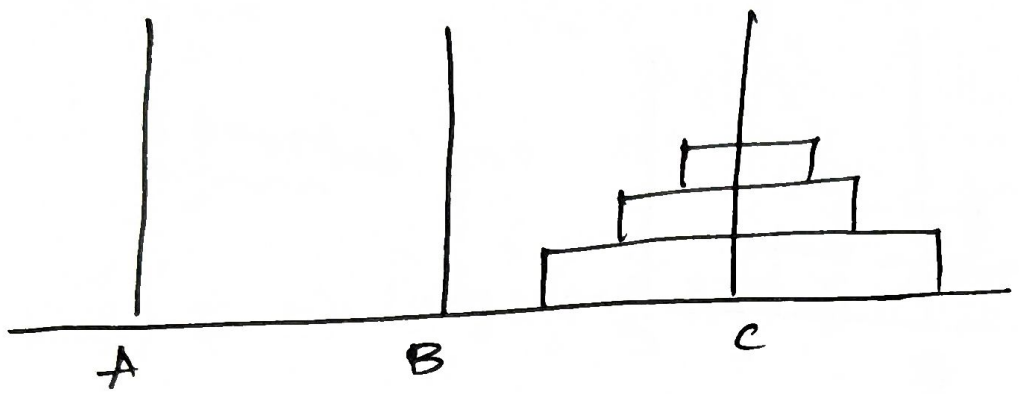
Move 4



Move 5



Move 6



Move 7

Solving the Recurrence :>

using substitution

$$T(n) = 2T(n-1)+1$$

Expand

$$T(n) = 2(2T(n-2)+1)+1$$

$$= 2^2 T(n-2) + 2 + 1$$

continue

$$= 2^k T(n-k) + (2^k - 1)$$

Put $K = n-1$

$$T(n) = 2^{n-1} T(1) + (2^{n-1} - 1)$$

Since $T(1) = 1$

$$T(n) = 2^{n-1} + 2^{n-1} - 1$$

$$T(n) = 2^n - 1$$

Final time complexity.

$$T(n) = 2^n - 1$$

$\therefore \quad T(n) = \Theta(2^n)$

Key observations.

* Exponential growth
* Very inefficient for large n
* Recursive depth = n
* No. of moves doubles with each additional disk.

Example values.

n (disks)	Moves T(n)
1	1
2	3
3	7
4	15
5	31

Conclusion:-

* Asymptotic notation helps analyze algorithm efficiency.

* Tower of Hanoi follows a recursive exponential pattern

* Its time complexity is: $\Theta(2^n)$.